



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SECP3843

SPECIAL TOPICS IN DATA ENGINEERING (WBL)

Semester 2 Session 2025/2026

Tutorial 4: Generative AI for Data Pipelines

Lecturer: Dr Aryati binti Bakri

Section: 01

NAME	MATRIC NO
NAJMA SHAKIRAH BINTI SHAHRULZAMAN	A23CS0140
NAWWARAH AUNI BINTI NAZRUDIN	A23CS014

Table of Contents

1.0 AI-Assisted Data Engineering.....	3
2.0 AI Agent Used In The Tutorial.....	3
3.0 Hands-On Steps in Airia.....	4
4.0 Results & Explanation.....	9
5.0 Reflection.....	16

1.0 AI-Assisted Data Engineering

AI-Assisted Data Engineering is the practice of integrating Artificial Intelligence (AI) into data pipeline workflows. As data grows in volume, velocity and variety, data preparation becomes exponentially difficult as each source may come in a different format, update at different rates and have varying quality standards (Chirumamilla, 2024). AI agents are able to detect quality issues such as missing values, duplicates, format inconsistencies, and outliers. This is especially useful for detecting semantically suspicious values entries that pass every programmatic check but are obviously wrong in context. For example, a traditional pipeline might fail to notice that a temperature reading of "-999" from a weather sensor is impossible, whereas an AI agent trained on historical sensor data would flag it as anomalous. This capability is particularly relevant to modern data engineering, where organizations increasingly rely on real-time streaming data from IoT devices, user clicks, and financial transactions which will require intelligent, adaptive handling rather than static scripts.

Figure 1.1 provides a flowchart on how AI can be integrated into data engineering workflow. The first step is to prepare the data and ensure that the data used is high-quality. Then the automated pipeline applies feature engineering, removes outliers, and does data imputation for clean and accurate inputs. Then, a suitable AI model will be selected depending on the context of the problem. Reinforcement learning is ideal for dynamic optimization problems such as resource allocation. Neural networks excel at complex pattern recognition in images, such as anomaly detection in sensor data. The models are then evaluated against metrics such as accuracy, latency, and recall to understand the performance of the model. Finally, the model is deployed and is monitored accordingly. It is a good practice to keep retraining the model as data evolves and monitor for any anomaly in the pipeline (Jonnalagadda et al., 2025).

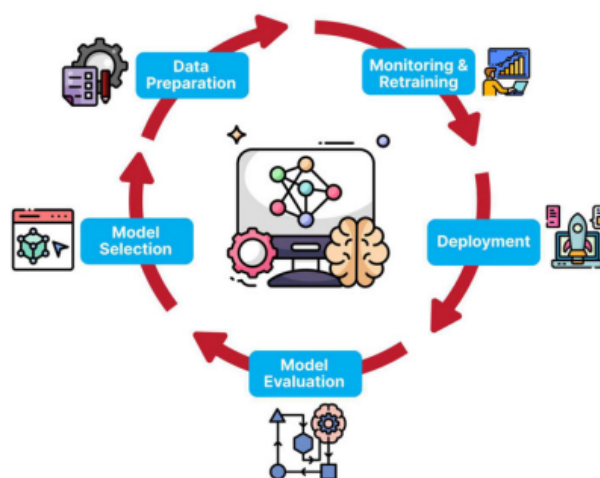


Figure 1.1 Flowchart of AI Implementation in Data Engineering (Jonnalagadda et al., 2025)

Several industries that are affected include the healthcare field where a global healthcare organization uses decentralized artificial intelligence to analyze medical data from different hospitals while still maintaining patient privacy. This has resulted in improvements for the accuracy of disease prediction models. A study by Kovvuri (2024), reveals that AI integrated solutions are able to achieve up to 67% reduction in data processing time, 89% enhancement in accuracy, and 45% decrease in operational costs. In short, AI-assisted data engineering simply works better for the scale and messiness of real-world data today.

2.0 AI Agent Used In The Tutorial

The tutorial demonstrates the use of Claude Haiku as the AI Model used within the Airia AI platform. Claude is a trustworthy and intelligent AI platform built by Anthropic. Figure 2.0 shows the use case diagram of the Airia AI Orchestration Platform.

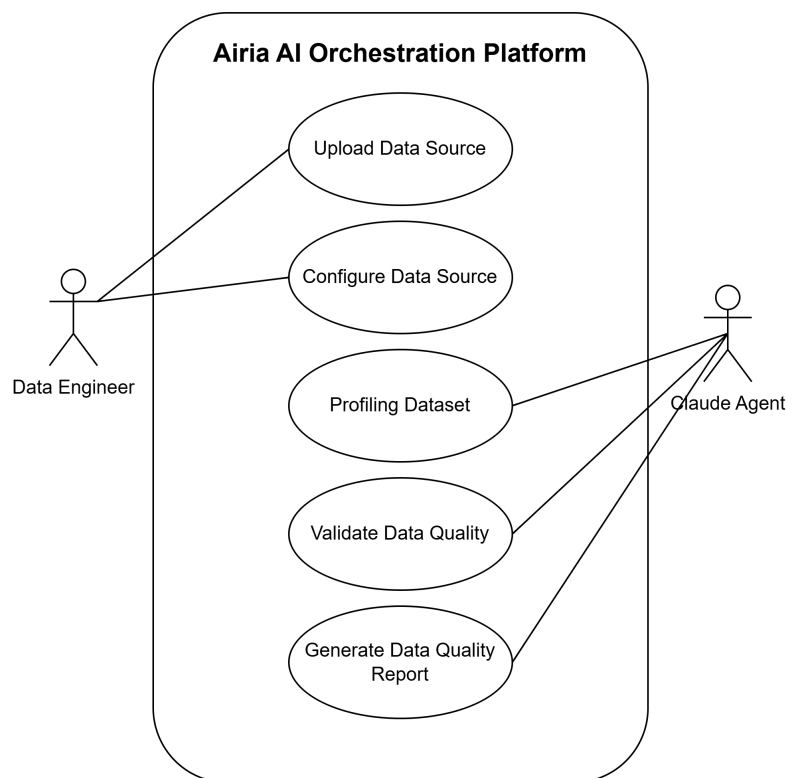


Figure 2.0 Use Case Diagram of Airia AI Orchestration Platform

In this tutorial, Claude holds three distinct uses:

1. **Data Profiling Agent:** After the Data Engineer uploads and configures the data source. Claude will receive the raw data and analyze each column for profiling. It will report what are

the data types for each column, null counts, and high-level attributes. Claude is automated to produce a structured profile without being explicitly scripted for a specific schema.

2. **Data Quality Validation Agent:** Claude is then instructed to identify quality issues by checking critical fields, duplicates, format inconsistencies, outliers, and semantically suspicious values. Claude's reasoning capability can flag unrealistic entries like a date field with value "9999-99-99". This would require extensive code to be able to catch if it was done manually.
3. **Report Generation Agent:** Receives the list of identified issues and produces a concise, human-readable data quality report summarising total records, records with issues, severity levels, the top five issues found, and recommended steps to be taken. This makes the pipeline's output immediately actionable for both engineers and business users.

3.0 Hands-On Steps in Airia

The data quality agent was built on Airia, a no-code AI agent platform. The pipeline was built by connecting steps on a visual canvas. Each step automatically passed its output to the next step. These steps formed a complete pipeline, from raw data ingestion to a finished data quality report.

Step 1: Build the Pipeline Structure

At the start, a template was already provided as shown in Figure 3.1. The first thing to do was to drag and drop the Data Source step onto the canvas. This step could be found after clicking the plus (+) button on the toolbar, and the File Upload option was dragged onto the canvas as shown in Figure 3.2. This step was used for data ingestion.

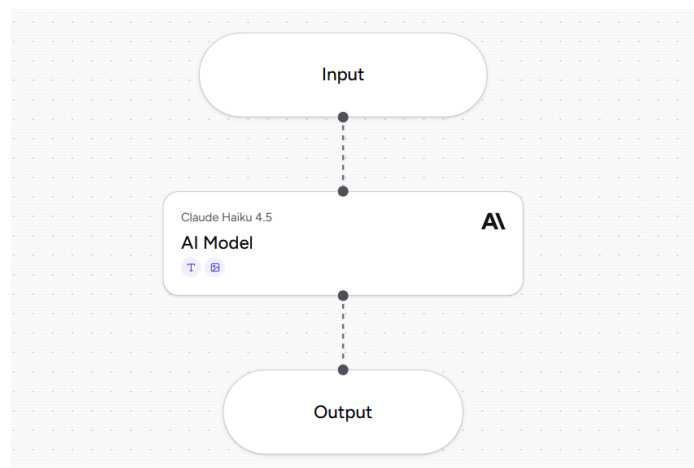


Figure 3.1 Original Template Found in Airia

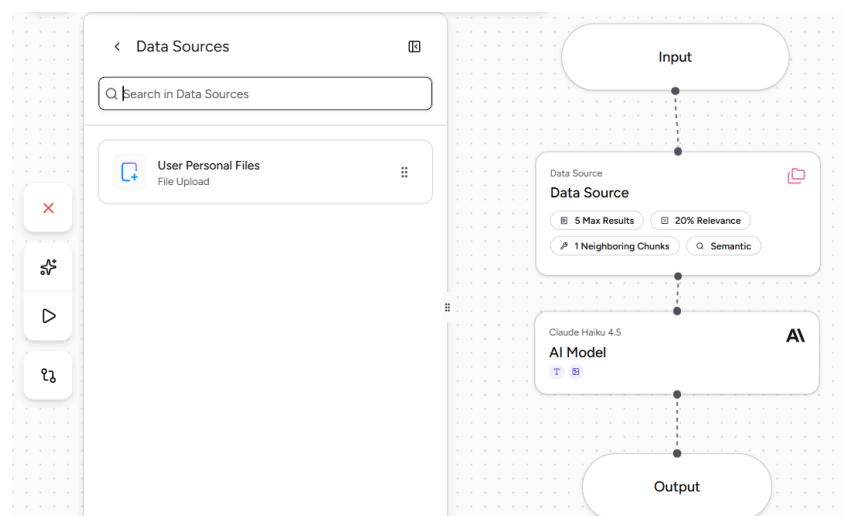


Figure 3.2 Adding the Data Source (File Upload) Step

Next, three more AI Model steps were dragged from the AI Models section onto the canvas. The first AI Model was used for data profiling, the second AI Model was used for data quality validation, and the last AI Model was used for generating the data quality report. The AI model used for this project is Claude Haiku 4.5 by Anthropic as shown in Figure 3.3.

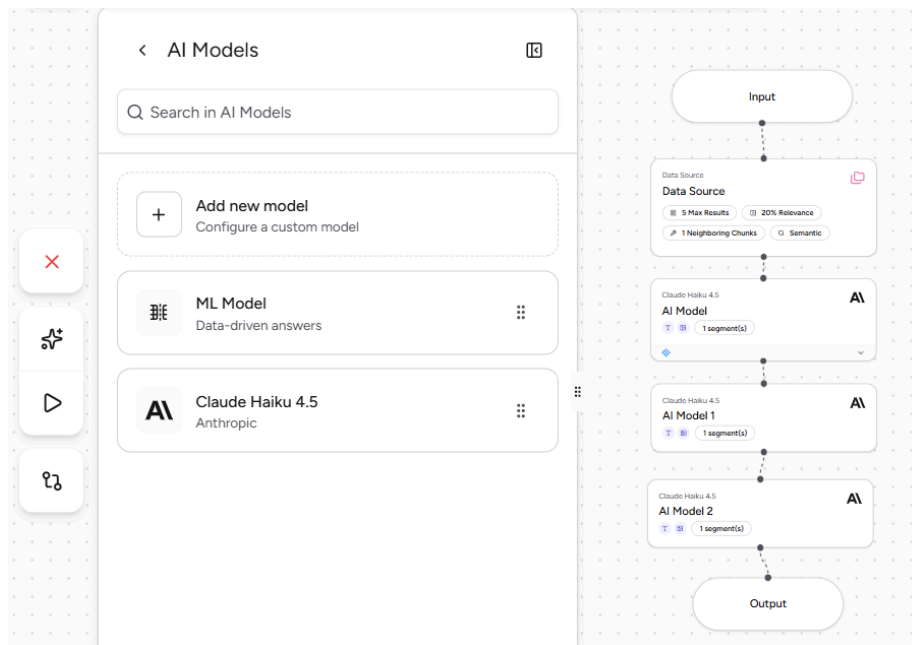


Figure 3.3 AI Model Added in the Pipeline

Step 2: Configure the Data Source Step for Data Ingestion

A CSV file about hotel bookings was then uploaded to the Data Source as shown in Figure 3.4. The Data Source step extracted data from the uploaded file using semantic search. It was configured to return a maximum of 5 results, with a relevance threshold of 20%. One neighbouring chunk was included for each result, so the surrounding context of each piece of data was not lost as shown in Figure 3.5. These settings were chosen to balance two things which are returning enough data for the AI models to work with, while avoiding results that were too unrelated to be useful. This step represented the data ingestion stage of the pipeline.

Data Sources

User Personal Files

File Upload

0 Failed

0 Pending

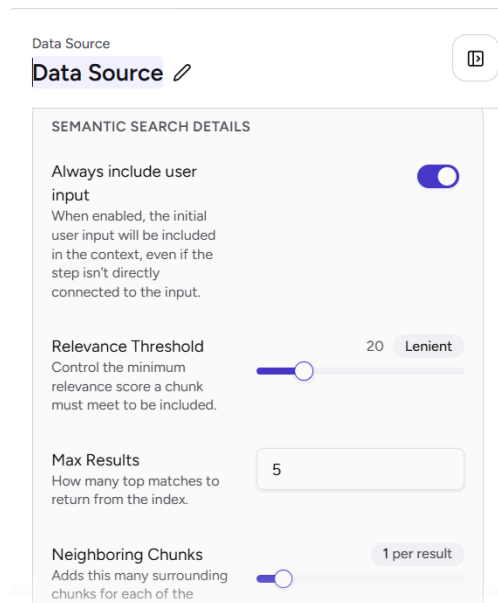
1 Ready

Q Search

Type

Add Files

	Name	Type	Size	Modified	Status	
	 hotel_bookings.csv	CSV	15.85 MB	Jun 15, 2026 10:52 AM	✓ Ready	...

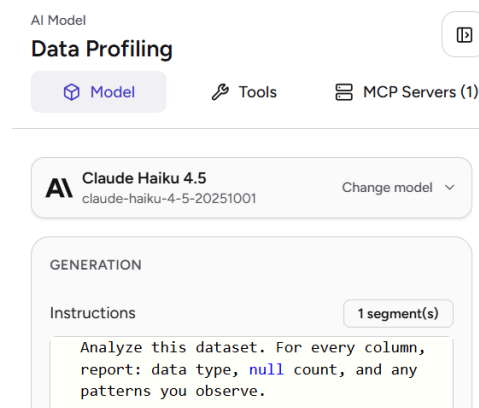


The image shows a configuration window for a 'Data Source'. At the top, there's a header 'Data Source' with an edit icon. Below it, a section titled 'SEMANTIC SEARCH DETAILS' contains several settings: 'Always include user input' is a toggle switch that is turned on; 'Relevance Threshold' is a slider set to 20 with a 'Lenient' label; 'Max Results' is a text input field containing the number 5; and 'Neighboring Chunks' is a slider set to 1 with a label '1 per result'.

Figure 3.5 Data Source Configuration

Step 3: Configure AI Model - Dataset Profiling (Prompt 1)

The first AI Model step used Claude Haiku 4.5 to profile the dataset. The prompt given to this step was: "Analyze this dataset. For every column, report: data type, null count, and any patterns you observe", as shown in Figure 3.6.



The image shows the configuration for an 'AI Model' step, specifically 'Data Profiling'. It has tabs for 'Model', 'Tools', and 'MCP Servers (1)'. The 'Model' tab is selected, showing 'Claude Haiku 4.5' with a 'Change model' dropdown. Below this, a 'GENERATION' section contains an 'Instructions' field with the text: 'Analyze this dataset. For every column, report: data type, null count, and any patterns you observe.' There is also a '1 segment(s)' label.

Figure 3.6 Prompt for AI Model Data Profiling

This is important first step because it provided the foundation for the rest of the pipeline. Before any data quality issue could be identified, the basic structure of the dataset needed to be understood first, such as the data type of each column, how many values were missing, and whether any unusual patterns existed. Without this profile, the later steps would have no reference point to judge whether a value was actually a problem or simply normal for that column.

Step 4: Configure AI Model 1 - Data Quality Issue Detection (Prompt 2)

The second AI Model used the column profile produced by the previous step as its input. The prompt given to this step was: "Based on the profile above, identify data quality issues including: null, missing values in fields, duplicate records, format inconsistencies, outliers, any semantically suspicious values. Classify each issue as Critical, Warning or Info," as shown in Figure 3.7. This step is part of the data transformation stage of the pipeline. It checks the column profile for data quality issues and labels each one based on how serious it is.

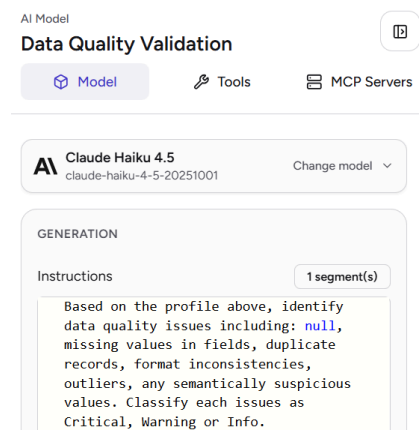


Figure 3.7 Prompt for AI Model Data Quality Validation

Step 5: Configure AI Model - Final Data Quality Report (Prompt 3)

The third AI Model generated the final report. The prompt given to this step was: "Generate a concise data quality report summarizing: total records, records with issue, breakdown by severity, top 5 issues with examples and a recommended step to fix it", shown in Figure 3.8.

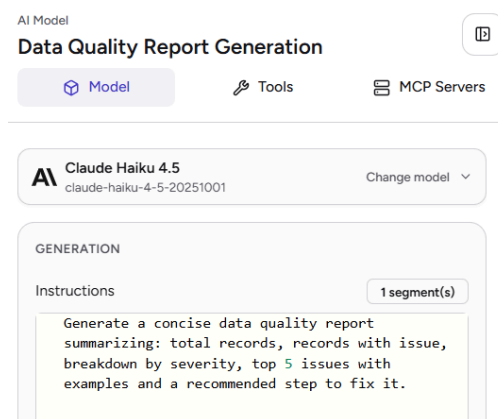


Figure 3.8 Prompt for AI Model Data Quality Report

Because chat history was enabled across all three AI Model steps, this final step could combine information from both the column profile and the issue list created earlier. The model did not need to repeat the analysis from scratch. Instead, it organised the earlier results into a final report. This step represents the final transformation stage of the pipeline, where raw findings were turned into a report that a person could read and act on.

Step 6: Full Data Pipeline

After all six steps were configured, the complete pipeline could be viewed as one connected flow on the canvas, as shown in Figure 3.9. The Input step received the uploaded dataset and passed it to the Data Source step for extraction. The extracted data then moved through the three AI Model steps in sequence. The first AI Model produced the column profile, the second AI Model used that profile to identify and categorise data quality issues, and the third AI Model turned those issues into the final data quality report. This report was then passed to the Output step, which displayed it to the user. This full flow showed how the pipeline carried out the entire process automatically from data ingestion and transformation to a finished report as the output.

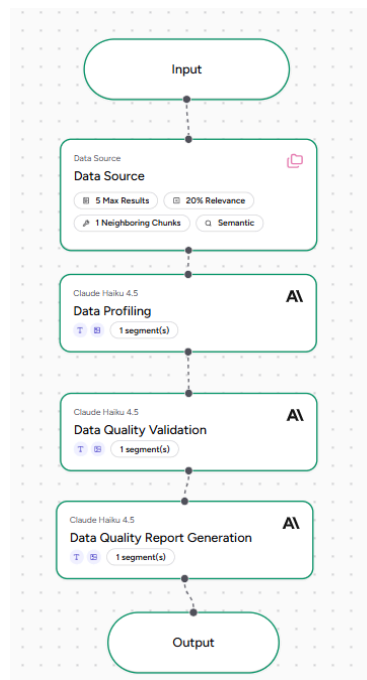


Figure 3.9 Full Data Quality Pipeline

4.0 Results & Explanation

To run the pipeline, a message such as 'run this data quality agent' was sent in the chat interface called Playground. Figure 4.1 shows the message used to start the pipeline. Once this message was sent, the pipeline ran automatically, moving through the Data Source step and then each of the three AI Model steps in sequence. The progress is shown in the right panel, as shown in Figure 4.2.



Figure 4.1 Prompt to Run the Pipeline

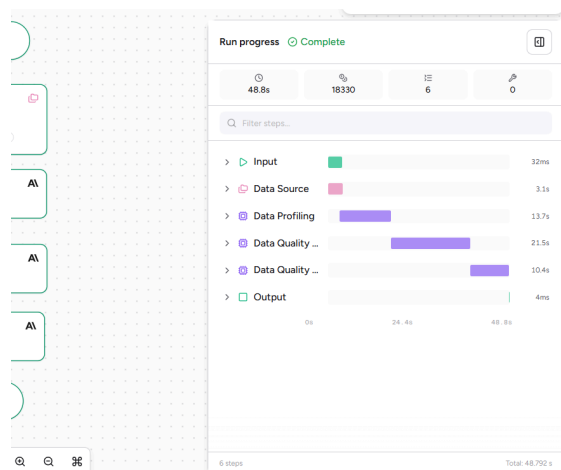


Figure 4.2 Progress Bar Shown at the Right Panel

Once the pipeline finished running, the execution log could be viewed, as shown in Figure 4.3. This log showed that all six steps completed successfully. The entire pipeline took 48.8 seconds to run and used a total of 18,330 tokens across the three AI Model steps, with no additional tool calls needed.

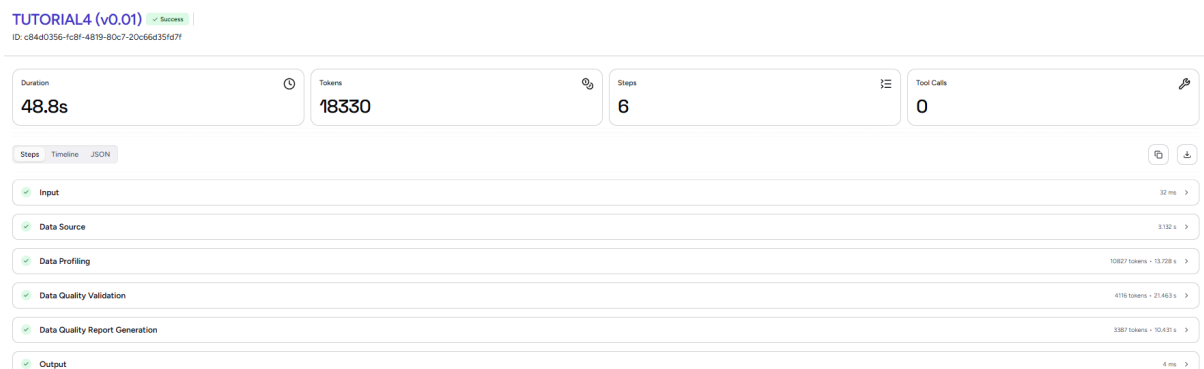


Figure 4.3 Execution Log

Each step in this log could be expanded individually to view its input and output of each step in detail. The first step is input. Figure 4.4 shows that the output for the input taken is the message that triggered this pipeline.

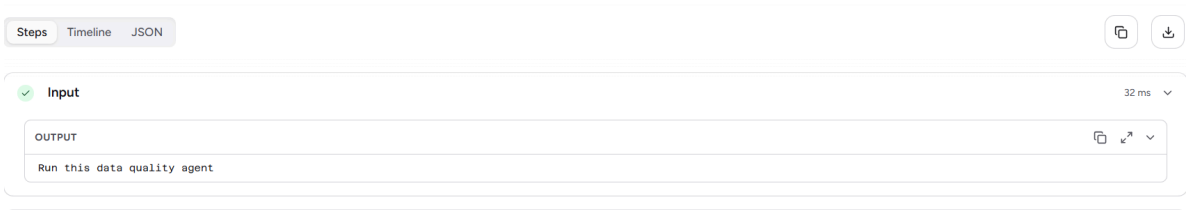


Figure 4.4 Input Expanded

Next, the Data Source step, Figure 4.5 show that it took the previous output as it processed data. The output for this step was retrieved in chunks containing data from hotel_bookings.csv.

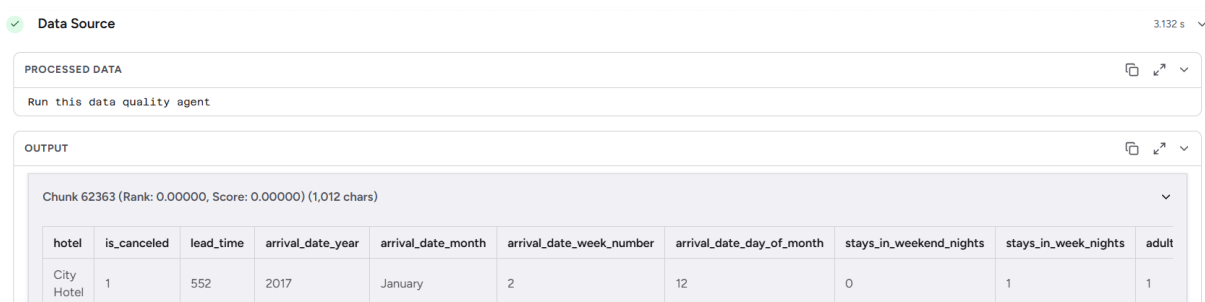


Figure 4.5 Data Source Expanded

Following that, the Data Profiling step used the retrieved chunk to analyse the dataset as it processed data as shown in Figure 4.6.

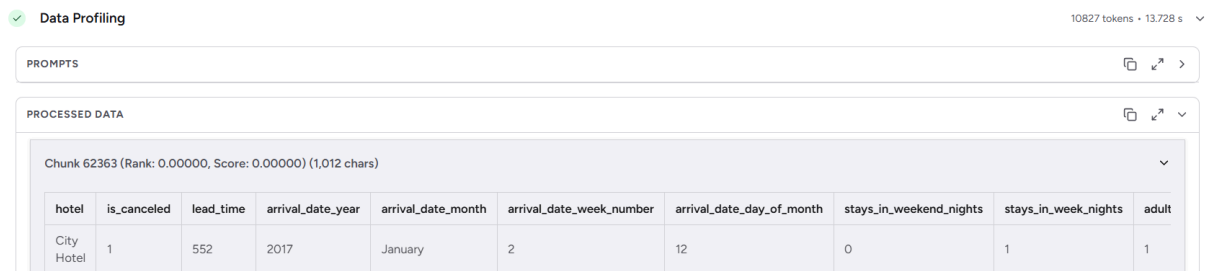


Figure 4.6 Data Profiling Expanded for Processed Data

The output is shown in Figure 4.7, where it does the column-by-column analysis of the dataset to find out the data type, null values, and pattern observations. The dataset shows a good finding where missing values are minimal. Continued in Figures 4.8 and 4.9, where it gives data quality findings about the dataset and recommendations. Overall, the system awards the dataset a quality score of 7.5.

OUTPUT

```
# Hotel Bookings Dataset - Data Quality Analysis

## Overview
Dataset: **hotel_bookings.csv**
Current Date: June 15, 2026
Records Analyzed: Sample of 14 chunks from the dataset

---

## Column-by-Column Analysis

| Column | Data Type | Null Count | Patterns & Observations |
|-----|-----|-----|-----|
| **hotel** | String/Categorical | 0 | All values: "City Hotel" (consistent single value in sample) |
| **is_canceled** | Integer/Binary | 0 | All values: 1 (100% canceled in sample - potential bias in sample selection) |
| **lead_time** | Integer | 0 | Range: 37-552 days; varies significantly (min 37, max 552) |
| **arrival_date_year** | Integer | 0 | All values: 2017 (single year only in sample) |
| **arrival_date_month** | String/Categorical | 0 | Value: "January" (consistent in sample) |
| **arrival_date_week_number** | Integer | 0 | Value: 2 (consistent in sample) |
| **arrival_date_day_of_month** | Integer | 0 | Values: 12, 14 (limited variety in sample) |
| **stays_in_weekend_nights** | Integer | 0 | All values: 0 (no weekend stays in sample) |
| **stays_in_week_nights** | Integer | 0 | All values: 1 (consistent 1-night stays) |
| **adults** | Integer | 0 | Values: 1, 3 (mostly 1 adult) |
| **children** | Integer | 0 | All values: 0 (no children in sample) |
| **babies** | Integer | 0 | All values: 0 (no babies in sample) |
| **meal** | String/Categorical | 0 | All values: "BB" (Bed & Breakfast only) |
| **country** | String/Categorical | 0 | All values: "PRT" (Portugal - single country) |
| **market_segment** | String/Categorical | 0 | Values: "Groups", "Online TA", "Corporate" (3 distinct types) |
| **distribution_channel** | String/Categorical | 0 | Values: "TA/TO", "Corporate" (2 distinct channels) |
| **is_repeated_guest** | Integer/Binary | 0 | All values: 0 (no repeat guests) |
| **previous_cancellations** | Integer | 0 | All values: 0 (no prior cancellations) |
| **previous_bookings_not_canceled** | Integer | 0 | All values: 0 (no prior bookings) |
| **reserved_room_type** | String/Categorical | 0 | Values: "A", "D" (room type codes) |
| **assigned_room_type** | String/Categorical | 0 | Values: "A", "D" (matches reserved in sample) |
| **booking_changes** | Integer | 0 | All values: 0 (no modifications to bookings) |
| **deposit_type** | String/Categorical | 0 | Values: "Non Refund", "No Deposit" (2 types) |
| **agent** | Integer | **2 NULL values** | Values: 1, 7, NULL (agent IDs or NULL for direct bookings) |
| **company** | Integer | **11 NULL values** | Values: 67, NULL (company/corporate IDs with many nulls) |
| **days_in_waiting_list** | Integer | 0 | All values: 0 (no waiting list delays) |
| **customer_type** | String/Categorical | 0 | All values: "Transient" (single type in sample) |
| **adr** | Float/Decimal | 0 | Range: 59-118.58 (average daily rate varies) |
| **required_car_parking_spaces** | Integer | 0 | All values: 0 (no parking requests) |
| **total_of_special_requests** | Integer | 0 | Values: 0, 1 (minimal requests) |
| **reservation_status** | String/Categorical | 0 | All values: "Canceled" (100% in sample) |
| **reservation_status_date** | Date | 0 | Format: DD-MM-YY; Range: 2015-2016 |

---
```

Figure 4.7 Output for Data Profiling

```
## Key Data Quality Findings

### ✅ Strengths
- **No missing values** in most columns (30/32 columns)
- **Consistent data entry** across hotel, meal type, country, and customer type
- **Valid date formats** in reservation_status_date
- **Binary fields** properly coded (0/1)

### ⚠️ Concerns & Anomalies

1. **Sample Bias** - All 14 records show `is_canceled = 1`
   - Suggests filtered/biased sample rather than population representation
   - Full dataset likely has canceled and non-canceled bookings

2. **Sparse Nulls** - Two columns with missing data:
   - `agent`: 2 NULLS (14.3%) - indicates direct bookings
   - `company`: 11 NULLS (78.6%) - primarily non-corporate bookings

3. **Zero Variance in Sample**
   - Single hotel type (City Hotel)
   - Single country (Portugal)
   - Single month (January)
   - Single customer type (Transient)
   - Indicates this is a filtered subset, not full dataset*
```

Figure 4.8 Output for Data Profiling Continuation

```
5. **Date Inconsistency**
- `arrival_date_year`: 2017
- `reservation_status_date`: 2015-2016
- "Status dates predate arrival dates" ⚠️ - Potential data quality issue

---

## Recommendations

| Priority | Issue | Action |
|-----|-----|-----|
| 🔴 High | Date inconsistency | Verify `reservation_status_date` logic and field definitions |
| 🔴 High | Sample representativeness | Confirm if analyzing full dataset or specific subset |
| 🟡 Medium | Null values in agent/company | Determine imputation strategy (0, "Direct", mode, etc.) |
| 🟡 Medium | Limited variance | If full dataset, explore distribution across all categorical values |
| 🟢 Low | Data completeness | Maintain current approach; minimal missing data |

---

## Data Quality Score: **7.5/10**
- Good completeness (94% non-null)
- Moderate consistency (high zero-variance, likely filtered sample)
- Potential logical inconsistencies (date ordering)
```

Figure 4.9 Output for Data Profiling Continuation

Then, the next step Data Quality Validation is performed, the processed data is taken from the previous output. Figure 4.10 shows the metrics of the validation engine pipeline, which successfully processed the 14 data chunks from hotel_bookings.csv.

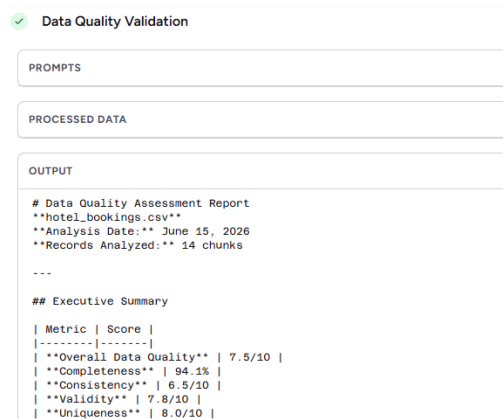


Figure 4.10 Output for Data Validation

Figure 4.11 shows the output for Data Validation Critical Issues, which explicitly highlights severe logical flaws like the chronological date conflict where reservation status changes predate the scheduled arrival years. Continued in Figures 4.12 and 4.13, where it details the warning and info-level issues, pinpointing the extreme sample skewness, such as the 100% cancellation rate and identifying expected, non-breaking null variations within the agent and corporate company ID fields.

```
**Status:** ⚠️ **CAUTION** - Multiple quality issues detected requiring investigation
---

## Detailed Issues Classification

### 🔴 CRITICAL ISSUES

#### 1. **Temporal Logic Error - Date Inversion**
- **Affected Fields:** 'arrival_date_year' vs 'reservation_status_date'
- **Issue:** Reservation status dates (2015-2016) are **1-2 years BEFORE** arrival dates (2017)
- **Severity:** CRITICAL
- **Impact:** Violates business logic - status cannot be recorded before reservation exists
- **Root Cause:** Likely data corruption, field mapping error, or reversed date formats
- **Recommendation:**
  - Verify field definitions and actual data types
  - Check if 'reservation_status_date' represents cancellation date vs. booking date
  - Audit data pipeline for ETL errors

#### 2. **Sample Selection Bias - All Canceled Bookings**
- **Affected Field:** 'is_canceled'
- **Issue:** 100% of sample (14/14 records) = 1 (canceled)
- **Severity:** CRITICAL
- **Impact:** Dataset appears pre-filtered for canceled bookings only
- **Business Risk:** Analysis results will not represent full population behavior
- **Recommendation:**
  - Confirm if analyzing full dataset or intentional subset
  - If full dataset: investigate data loading/filtering logic
  - Document sampling methodology clearly
```

Figure 4.11 Output for Data Validation Critical Issues

```

### 🟡 WARNING ISSUES

#### 4. **Suspicious Null Distribution in Agent Field**
- **Affected Field:** `agent`
- **Issue:** 2 of 14 records NULL (14.3%)
- **Severity:** WARNING
- **Semantic Issue:** NULLs likely represent direct bookings (no travel agent), but not explicitly coded
- **Impact:** Moderate - affects agent/channel analysis
- **Recommendation:**
  - Code as 0 or "Direct" instead of NULL for clarity
  - Verify correlation with `distribution_channel` field
  - Document NULL meaning in data dictionary

#### 5. **Extremely Limited Sample Variance**
- **Affected Fields:** Multiple categorical fields
- **Issues:**
  | Field | Unique Values | Coverage |
  |-----|-----|-----|
  | hotel | 1 | 100% (City Hotel only) |
  | country | 1 | 100% (Portugal only) |
  | arrival_date_month | 1 | 100% (January only) |
  | customer_type | 1 | 100% (Transient only) |
  | meal | 1 | 100% (BB only) |
  | is_repeated_guest | 1 | 100% (0 only) |
- **Severity:** WARNING
- **Impact:** Cannot evaluate:
  - Multi-hotel performance
  - International vs. domestic patterns

```

Figure 4.12 Output for Data Validation Warning Issues

```

### 🟢 INFO LEVEL ISSUES

#### 7. **Room Type Consistency Check (PASS)**
- **Affected Fields:** `reserved_room_type` vs `assigned_room_type`
- **Issue:** No room type mismatches observed in sample (A→A, D→D)
- **Severity:** INFO
- **Note:** This is actually positive data quality indicator
- **Observation:** All cancellations occurred before room assignment mattered

#### 8. **Lead Time Outliers Within Normal Range**
- **Affected Field:** `lead_time`
- **Range:** 37-552 days (513-day spread)
- **Severity:** INFO
- **Assessment:** Range appears reasonable (last-minute to 18+ months advance)
- **Recommendation:** Monitor for extreme outliers (>1000 days) in full dataset

#### 9. **ADR (Average Daily Rate) Moderate Variation**
- **Affected Field:** `adr`
- **Range:** 59.00 - 118.58 EUR
- **Severity:** INFO
- **Assessment:** Reasonable hotel pricing range
- **Note:** All rates are non-negative (good data validation)

```

Figure 4.13 Output for Data Validation Info Level Issues

Finally, Figure 4.14 shows the output for Data Validation Matrix and Recommended Next Steps. This operational matrix maps out the severity profile of the data anomalies to provide a clear path forward that advises an immediate audit of the data logging workflow.

```

## Data Quality Matrix
| Category | Issue | Count | Severity | Status |
|-----|-----|-----|-----|-----|
| **Completeness** | Null values | 13 | CRITICAL | Requires review |
| **Consistency** | Temporal contradictions | 1 | CRITICAL | Blocking analysis |
| **Validity** | Logical contradictions | 1 | CRITICAL | Blocking analysis |
| **Uniformity** | Zero variance fields | 8+ | WARNING | Limits analysis scope |
| **Accuracy** | Potential bias (100% canceled) | 1 | CRITICAL | Sampling issue |

---

## Recommended Next Steps

### 🟡 Immediate Actions Required

1. **VERIFY DATASET SCOPE**
...
SELECT COUNT(*), is_canceled, COUNT(DISTINCT hotel), COUNT(DISTINCT country)
FROM hotel_bookings
...
Confirm if analyzing full dataset or filtered subset

2. **INVESTIGATE DATE FIELDS**
- Check data dictionary for `reservation_status_date` definition
- Run: `SELECT MIN(arrival_date_year), MAX(reservation_status_date)`
- Determine if dates are reversed or field semantics misunderstood

```

Figure 4.14 Output for Data Validation Matrix and Recommended Next Step

Following validation, the system moves to the Data Quality Report Generation and Output step. For this process, the report generation engine takes the validated data from the previous output as its input. It then processes these findings to compile the final assessment. Finally, the output step takes this completed report and delivers it directly to the user via the chatbot interface. Therefore, while it represents a distinct pipeline step focused on formatting and delivery, the underlying analytical data remains consistent with the previous validation findings. As shown in Figure 4.15, the final data quality report indicates an overall data quality score of 7.5 out of 10, suggesting that the dataset has a generally acceptable level of quality but still contains several issues that require attention. The report shows that the majority of the data passed the validation checks. It also categorises the issues by their severity critical, warning and informational.

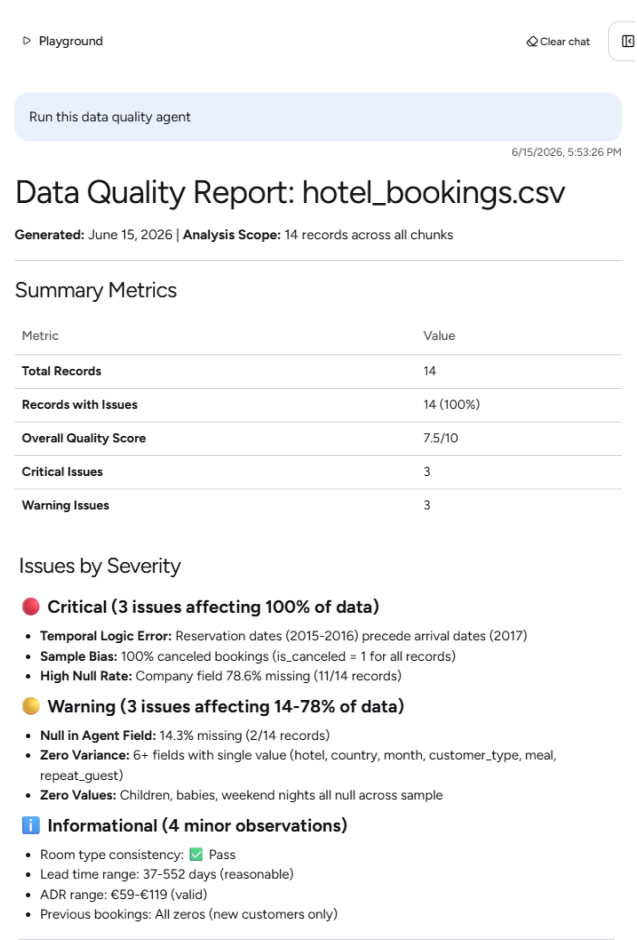


Figure 4.15 Final Report Data Quality Produced

The report also provides a breakdown of the detected issues. As shown in Figure 4.16 below, the most common problem identified was missing values, which occurred most frequently among all detected issues. Other significant issues included duplicate records, invalid data formats, inconsistent values, and outliers. These issues were ranked in the Top 5 Issues section based on their occurrence and

impact on overall data quality. The presence of missing values and duplicate records contributed significantly to the reduction of the overall quality score.

Top 5 Issues with Examples				
#	Issue	Records Affected	Example	Fix
1	Date Inversion	14/14 (100%)	Arrival: 2017-01-01, Status Date: 2015-12-01	Verify field definitions in data dictionary; check ETL field mapping; may need date reversal
2	100% Canceled Bookings	14/14 (100%)	All is_canceled = 1	Query full table: <pre>1 SELECT COUNT(*), is_canceled FROM hotel_bookings GROUP BY is_canceled</pre> to confirm if intentional subset
3	Missing Company Data	11/14 (78.6%)	company = NULL (78.6% of records)	Recode NULL as "Direct" or "0"; distinguish missing from no-company; update data dictionary
4	Single Hotel Only	14/14 (100%)	hotel = "City Hotel" for all	Expand sample to include "Resort Hotel"; verify if full dataset has both hotels
5	Single Month Only	14/14 (100%)	arrival_date_month = "January" for all	Include all 12 months in analysis; verify data extraction logic wasn't filtered by month

Figure 4.16 Final Report Produced Top 5 Issues

To help improve the dataset, the system generated several recommendations, as shown in Figure 4.17. The recommendations suggest actions such as filling in missing values, removing duplicate records, standardizing data formats, validating input values against predefined rules, and reviewing abnormal records identified as outliers.

Recommended Fix Steps

Priority 1: Blocking Issues (Do First)

```
1 1. Validate date field semantics with data owner
2 2. Confirm if analyzing full dataset or intentional filtered subset
3 3. Investigate ETL pipeline for unintended filtering or field mapping errors
```

Priority 2: Data Standardization (Do Second)

```
1 4. Recode NULL values: agent NULL → "Direct", company NULL → "Unknown"
2 5. Separate analysis by booking status (canceled vs. completed)
3 6. Document all NULL meanings in data dictionary
```

Priority 3: Scope Expansion (Do Third)

```
1 7. Re-extract data to include all 12 months, both hotel types, all customer segments
2 8. Validate sample represents full population distribution
3 9. Segment analysis by repeat vs. new customers, corporate vs. direct
```

Recommendation

DO NOT USE FOR ANALYSIS until critical issues addressed. Estimated remediation: **2-4 hours**.
Currently suitable only for EDA with documented caveats.

🌐 18,330 \$ 0.03942634 ⌚ 48,853ms
6/15/2026, 5:54:07 PM

View Logs Realtime Execution

Figure 4.17 Final Report Produced Recommendations

The output for this data quality pipeline indicates that while the dataset is usable, improvements are needed to resolve the identified quality issues and increase the data quality score. The generated recommendations can assist users in prioritizing corrective actions to enhance data consistency, completeness, and accuracy.

5.0 Reflection

This section presents our reflections and insights gained from completing this tutorial.

Nawwarah Auni binti Nazrudin

This tutorial demonstrated the growing importance of AI in modern data engineering practices. Through the development of a data quality pipeline, a clearer understanding was gained of how AI can automate complex analytical tasks. The use of Airia as a no-code platform made it easier to understand the flow of data from ingestion to output. The tutorial also highlighted the importance of prompt design, as the quality of the generated outputs depended on the clarity of the instructions provided to the AI model. The results generated by Claude Haiku 4.5 showed that AI can effectively identify data quality issues, classify their severity, and recommend suitable corrective actions. Furthermore, the experience provided practical exposure to common data quality issues, including missing values, duplicates, and inconsistent formats, as well as the methods used to detect and address them.

Najma Shakirah binti Sharulzaman

This tutorial demonstrated the value of integrating Claude into modern data engineering workflows by shifting rule-based validation to an automated data quality pipeline. Developed on a no-code platform, the visual canvas of Airia AI platform makes it seamless to map the entire flow of data from ingestion to output through sequentially linked reasoning nodes. The pipeline orchestrates data smoothly across distinct operational steps: starting with foundational dataset profiling, moving into a deep data quality validation phase to detect and categorize anomalies by severity, and concluding with automated report generation. Because chat history is explicitly enabled across the workflow, subsequent Claude nodes preserve contextual information and use earlier analytical findings automatically without repeating processing from scratch. Ultimately, this hands-on experience highlights how embedded models like Claude can effectively automate complex data engineering tasks, while still reinforcing that the overall accuracy and depth of the generated metrics.

References

- Chirumamilla, K. R. (2024). Autonomous AI system for End-to-End data engineering. *International Journal of Intelligent Systems and Applications in Engineering*.
<https://doi.org/10.17762/ijisae.v12i13s.7964>
- Jonnalagadda, A. K., Dutta, K. P., Ranjan, P., & Myakala, P. K. (2025). AI and Optimization: Transforming Data Engineering Applications. In *Advances in intelligent systems research/Advances in Intelligent Systems Research* (pp. 686–702).
https://doi.org/10.2991/978-94-6463-787-8_52
- Kovvuri, V. K. R. (2024). The role of AI in data engineering and integration in cloud computing. *International Journal of Scientific Research in Computer Science Engineering and Information Technology*, 10(6), 616–623.
<https://doi.org/10.32628/cseit241061103>